

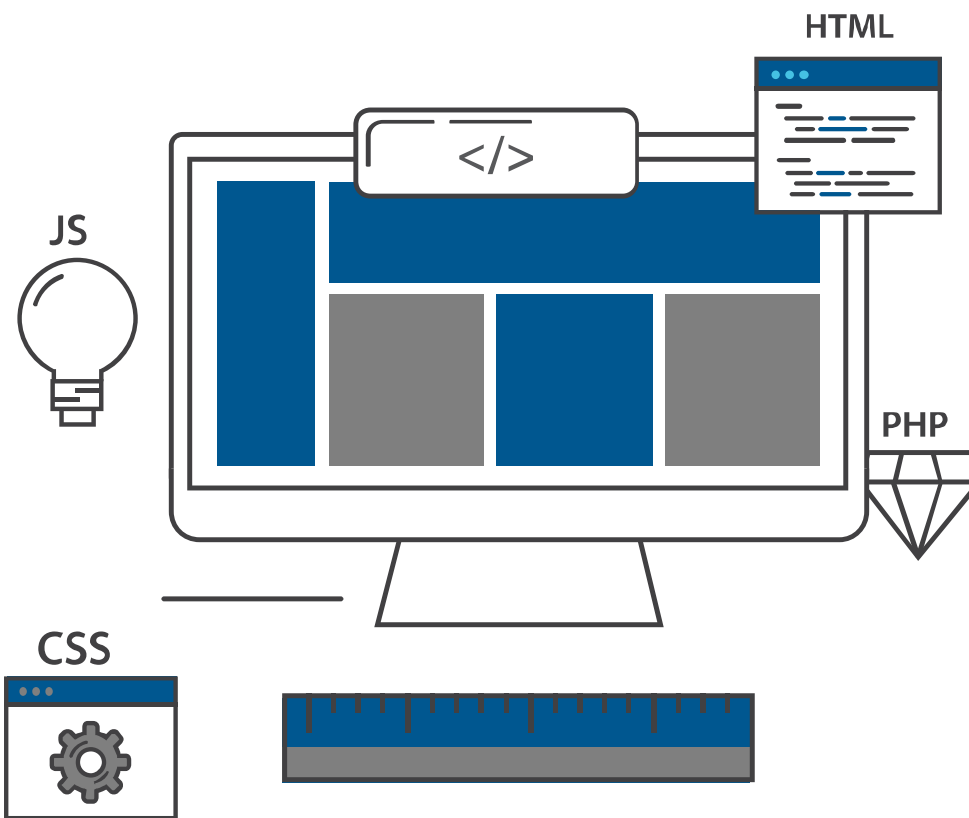
第七章 网络流

李翔

<https://implus.github.io/>



C++



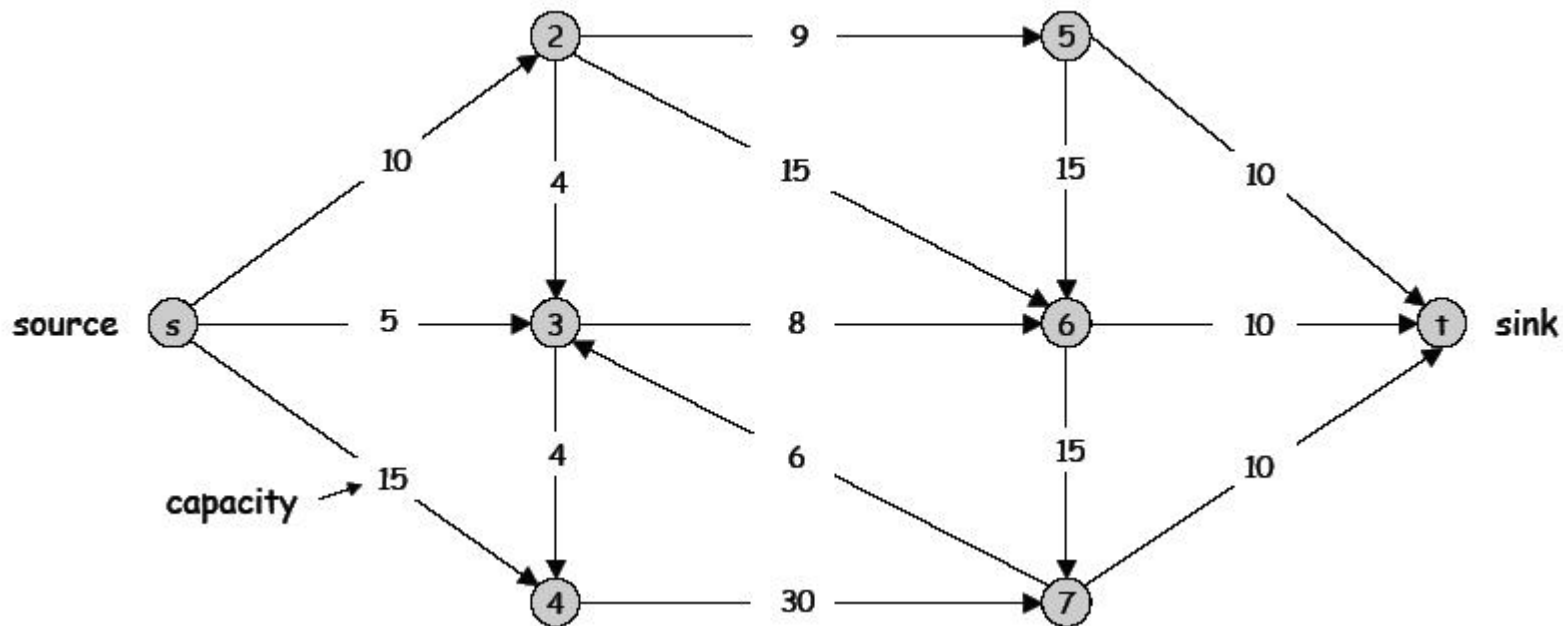
最大流问题

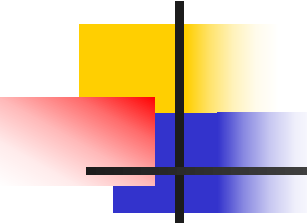
流网络

有向图 $G=(V,E)$

每条边关联一个容量, 非负数 C_e .

存在单一源点 s , 以及单一汇点 t

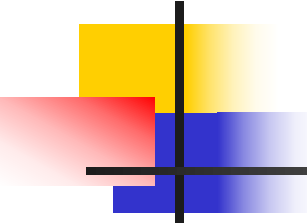




最大流问题

- **流**的定义:
- **s-t流**是一个函数**f**,它把每条边**e**映射到一个非负实数 $f: E \rightarrow R^+$, 值**f(e)**表示由边**e**携带的流量, 一个流**f**必须满足下面两个性质:
 1. (容量条件) $0 \leq f(e) \leq C_e$
 2. (守恒条件)除了**s,t**外, 对每个结点**v**,满足

$$\sum_{e_{in_v}} f(e) = \sum_{e_{out_v}} f(e)$$

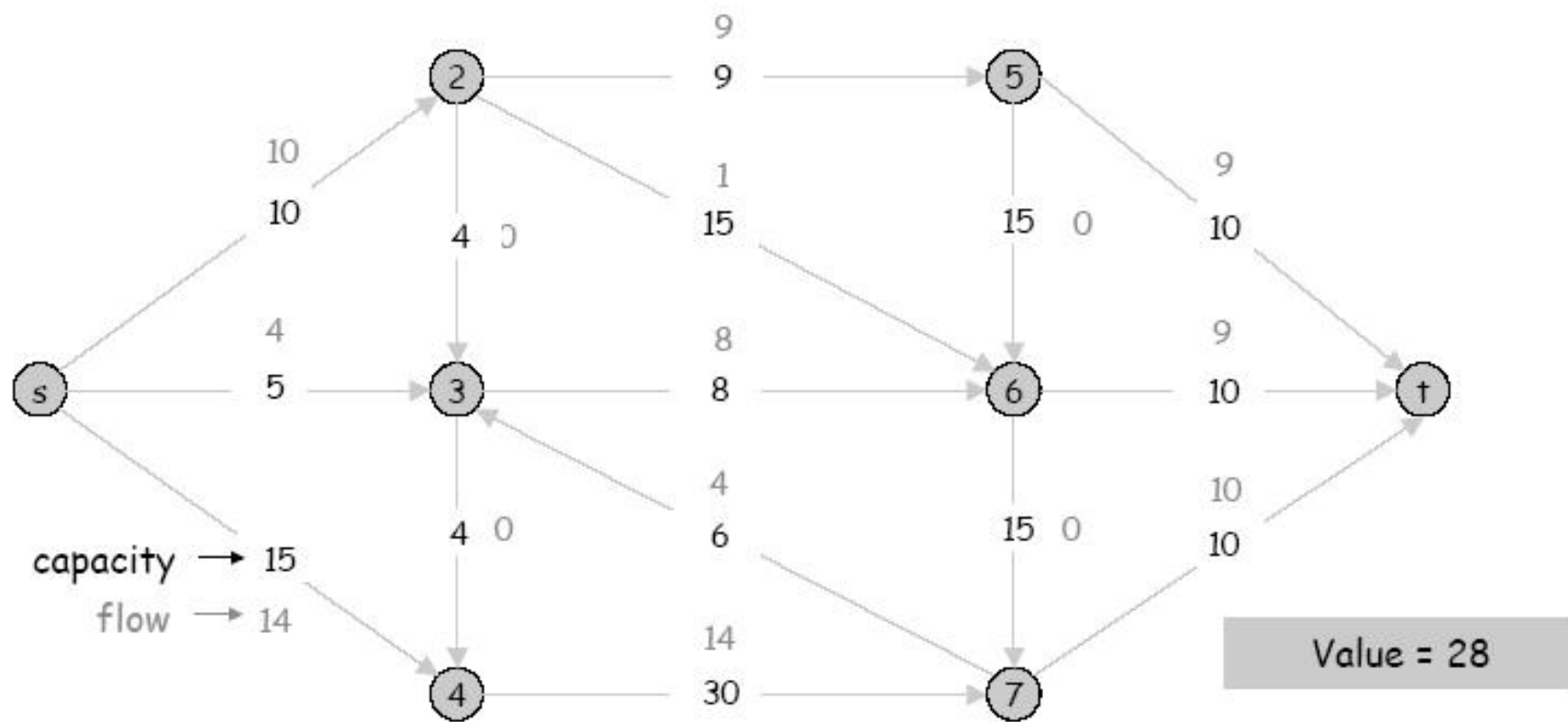


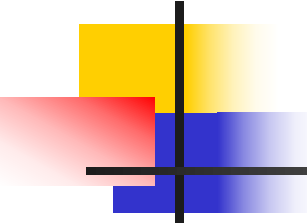
最大流问题

- 定义 $f^{out}(v) = \sum_{e_{out_v}} f(e), f^{in}(v) = \sum_{e_{in_v}} f(e)$
- 我们记 $v(f) = f^{out}(s)$
- **最大流问题**: 给定一个流网络, 自然的目标就是安排交通使得有效容量尽可能得到有效使用: 找出一个具有最大值的流

最大流问题

实际上最大的流:





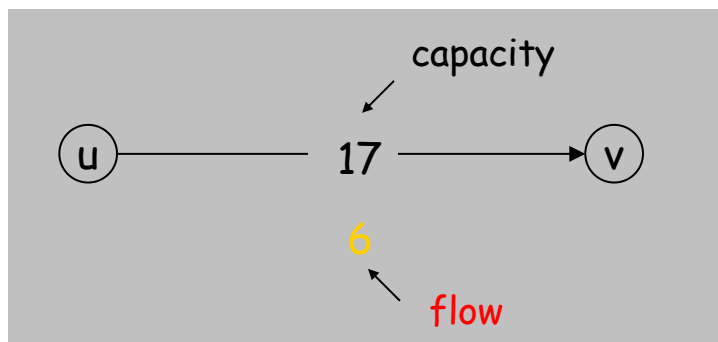
最大流问题

- 解决问题的思路
 - 分解
- 流的性质
- 我们可以在边上用剩余的容量**向前推**，并且我们可以在已经有流的边上**向后推**，使它转向一个不同的方向。
- 下面将引出**剩余图**的概念

最大流问题

■ 原始边

$e = (u, v) \in E$, 流 $f(e)$, 容量 $c(e)$.



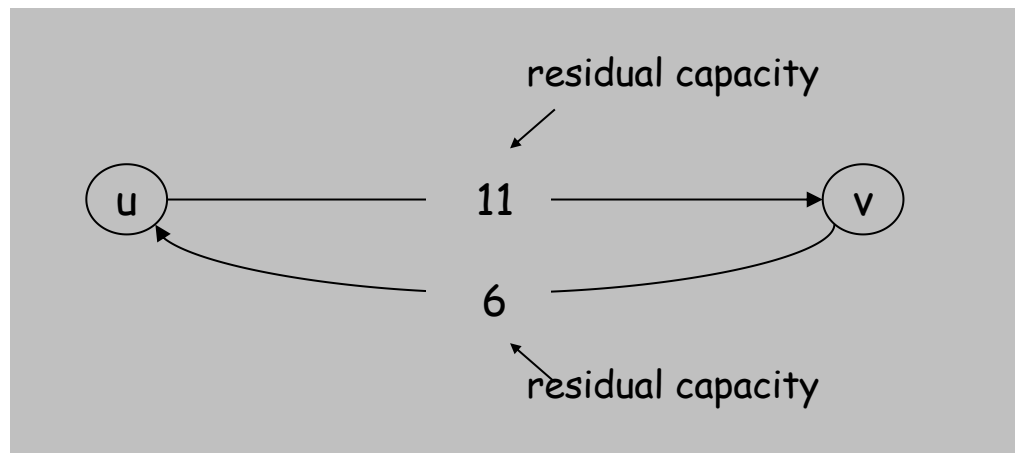
■ 剩余边

$e = (u, v)$ and $e^R = (v, u)$.

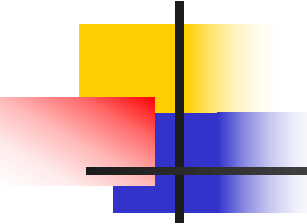
剩余容量:

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{for } e \\ f(e) & \text{for } e^R \end{cases}$$

最大流问题



- 剩余图: $G_f = (V, E_f)$.
 - 具有正的剩余容量的剩余边.
 - $E_f = \{e\} \cup \{e^R\}$.



最大流问题

- 对 G 的每条边 $e=(u,v)$,其中 $f(e) < c(e)$,那么存在 $c(e)-f(e)$ 的剩余的容量,我们还可以尝试在这个容量往前推,于是 G_f 中包含这条边 e ,容量为 $c(e)-f(e)$,称为前向边。
- 对 G 的每条边 $e=(u,v)$,其中 $f(e) > 0$,我们可以通过向后推这个流来“撤销”它,于是 G_f 中包含边 $e'=(v,u)$,容量是 $f(e)$,称为后向边。

最大流问题

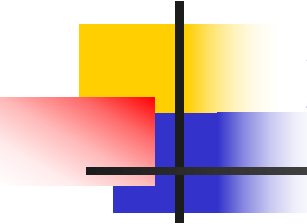
在剩余图中的增广路径

- 令 P 是 G_f 中一条简单的 s - t 路径。定义 $\text{bottleneck}(P, f)$ 是 P 上任何边关于流 f 的 **最小剩余容量**。如下算法 $\text{augment}(f, P)$ 在 G 中产生一个新的流 f' 。 ($f \rightarrow f'$)

```
Augment(f, P) {  
    b  $\leftarrow$  bottleneck(P)  
    foreach  $e \in P$  {  
        if ( $e \in E$ )  $f(e) \leftarrow f(e) + b$   
        else  $f(e^R) \leftarrow f(e) - b$   
    }  
    return f  
}
```

前向边

后向边



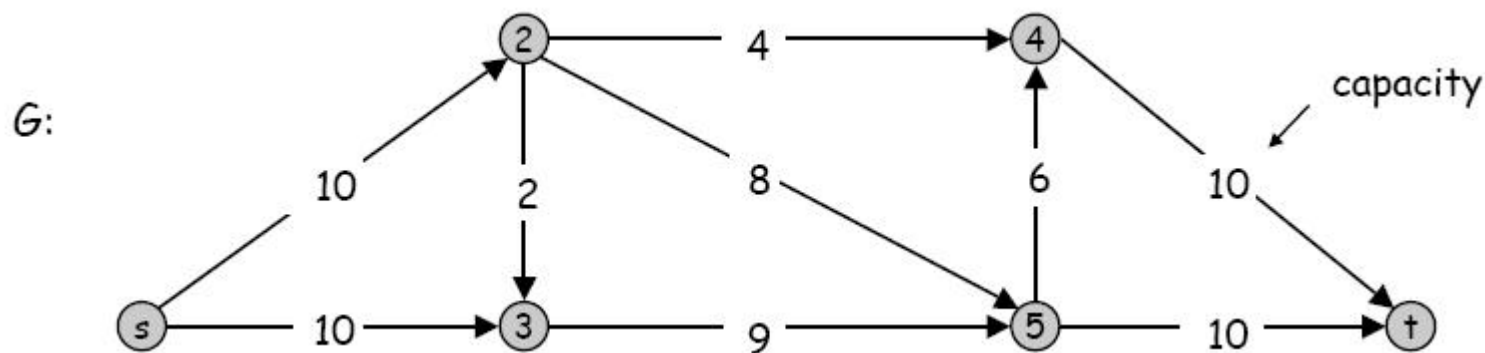
算法设计

- 增广操作保持了向前和向后流的守恒性
- 直觉上告诉我们，可以不断调整 G_f 来获取更大的流量。

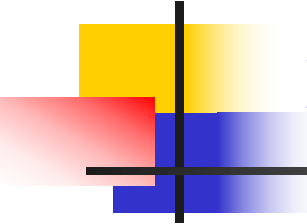
```
Ford-Fulkerson( $G, s, t, c$ ) {  
    foreach  $e \in E$   $f(e) \leftarrow 0$   
     $G_f \leftarrow$  residual graph  
  
    while (there exists augmenting path  $P$ )  
    {  
         $f \leftarrow$  Augment( $f, c, P$ )  
        update  $G_f$   
    }  
    return  $f$   
}
```

算法分析

- Demo演示(增广路径、瓶颈容量、剩余网络、 $f(x,y)$ 最大流)
- 初始图G

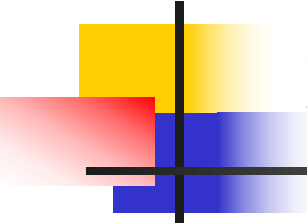


https://blog.csdn.net/qq_38662930/article/details/105242945



算法分析

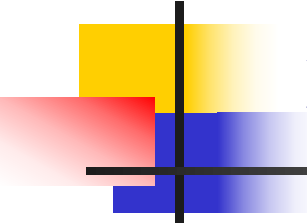
- 最大可能的流值: $v(f) \leq C = \sum_{e \text{ out of } s} c_e$
- 因为有一个上界, 我们知道Ford-Fulkerson算法一定会终止
- 定理7.4 如上所述, 假设在流网络G中的所有容量都是整数。那么Ford-Fulkerson算法在至多C次While循环的迭代后终止。



算法分析

下面考虑Ford-Fulkerson算法的运行时间。

- n 表示 G 中的结点数, m 表示 G 中的边数
- 算法复杂度应该是?
- 定理7.5 假设在流网络 G 中的所有容量都是整数, 那么Ford-Fulkerson算法可以在 $O(mC)$ 时间内实现



算法分析

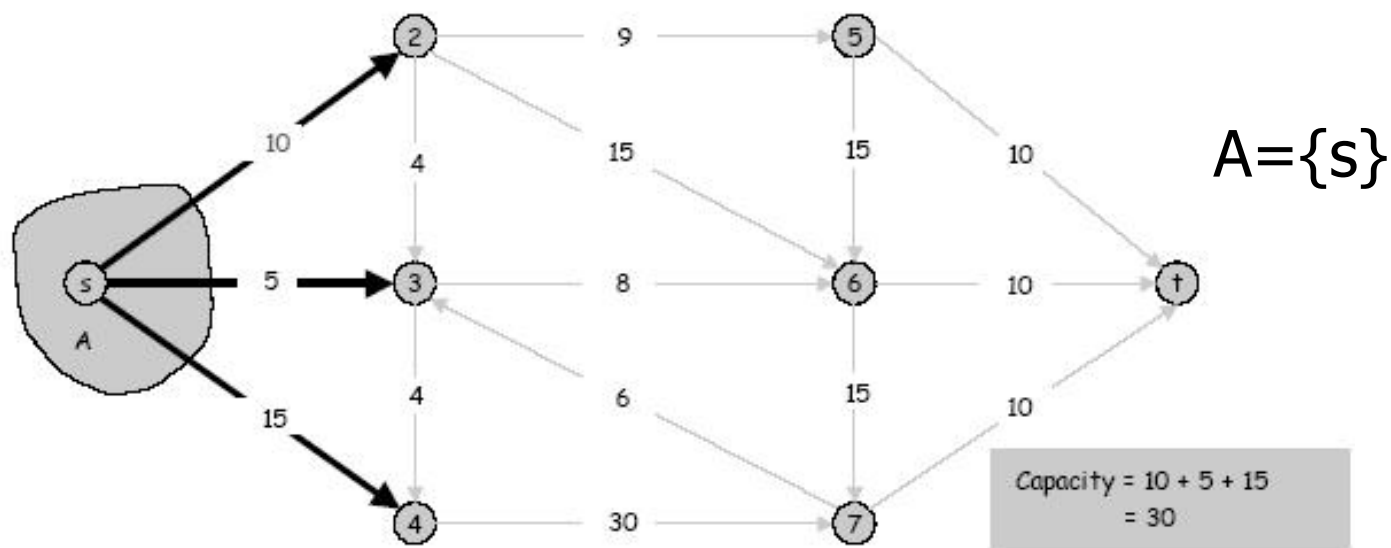
■ 证明:

我们知道While循环至多在C次迭代后终止。于是考虑流调整一次时需要的复杂度:

- 剩余图至多有 $2m$ 条边, 为找到 G_f 中一条 $s-t$ 路径, 可以考虑宽度优先或者广度优先搜索, 代价为 $O(m+n)=O(m)$;
- 因为路径 P 至多有 $n-1$ 条边, 建立新的剩余图需要 $O(m)$ 时间。

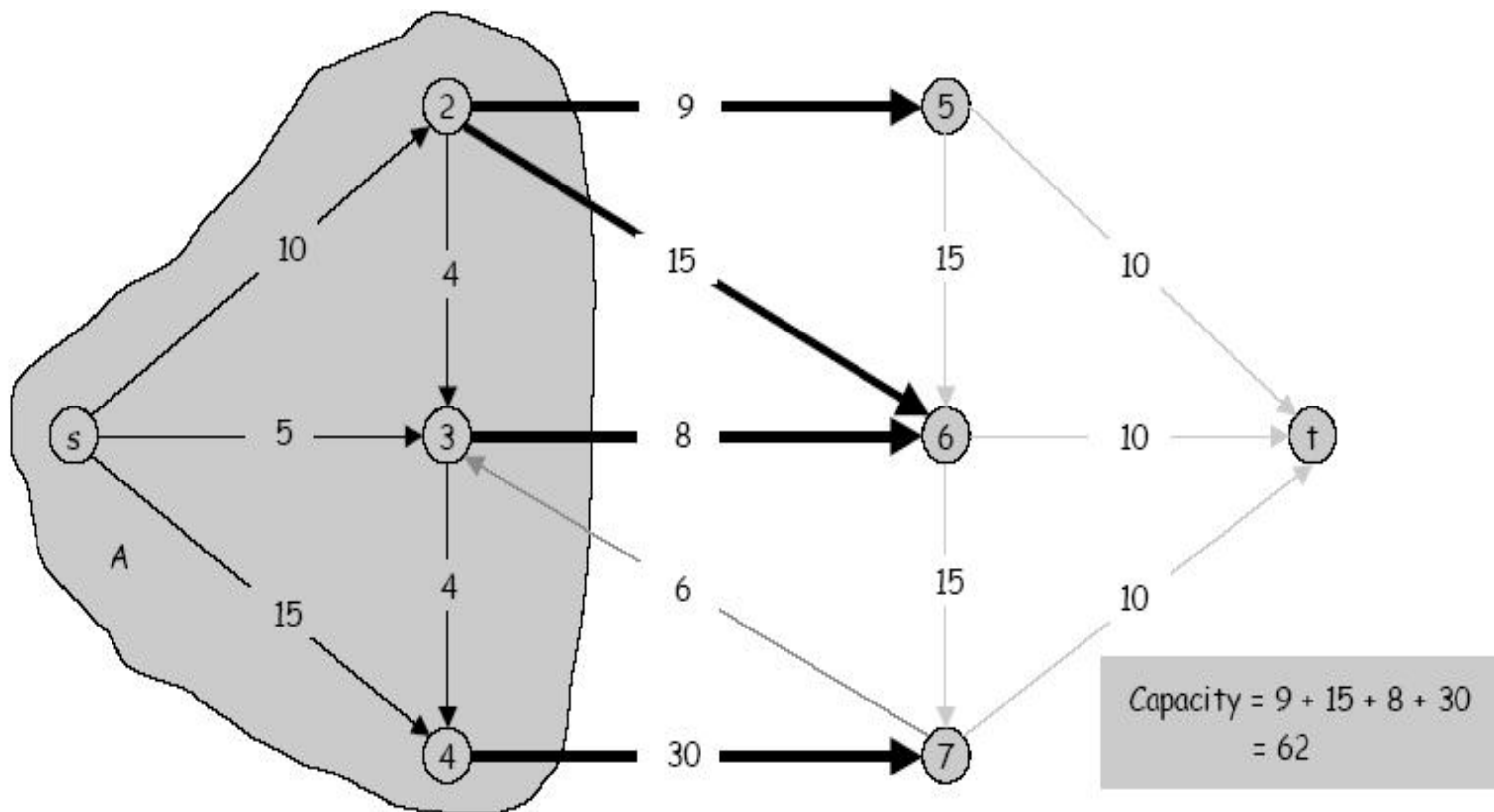
7.2 网络中的最大流与最小割

- 我们说一个s-t割是结点集合V的一个划分(A,B), 使得 $s \in A, t \in B$. 一个割(A,B)的容量记为 $c(A,B)$. 也就是从A出来的所有边的容量之和。



最大流与最小割

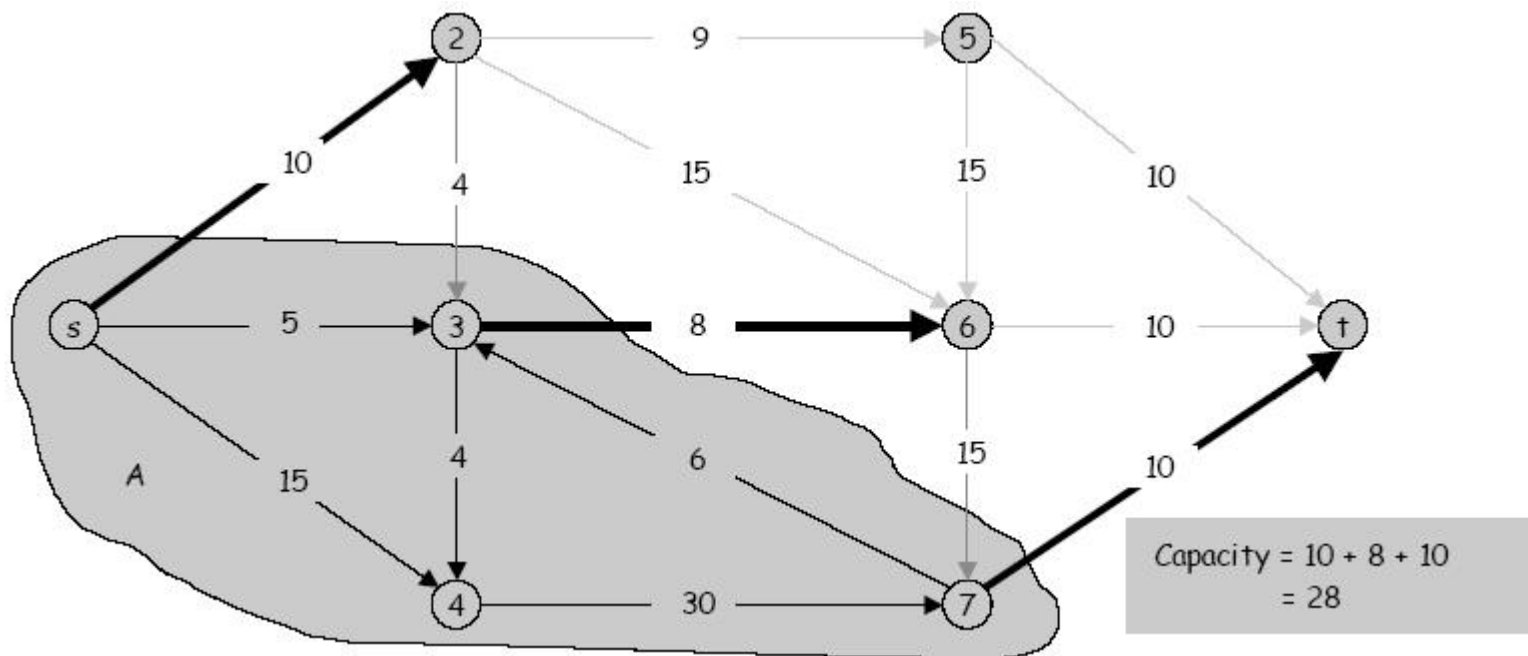
另外一种割的划分: $A=\{s,2,3,4\}$



最大流与最小割

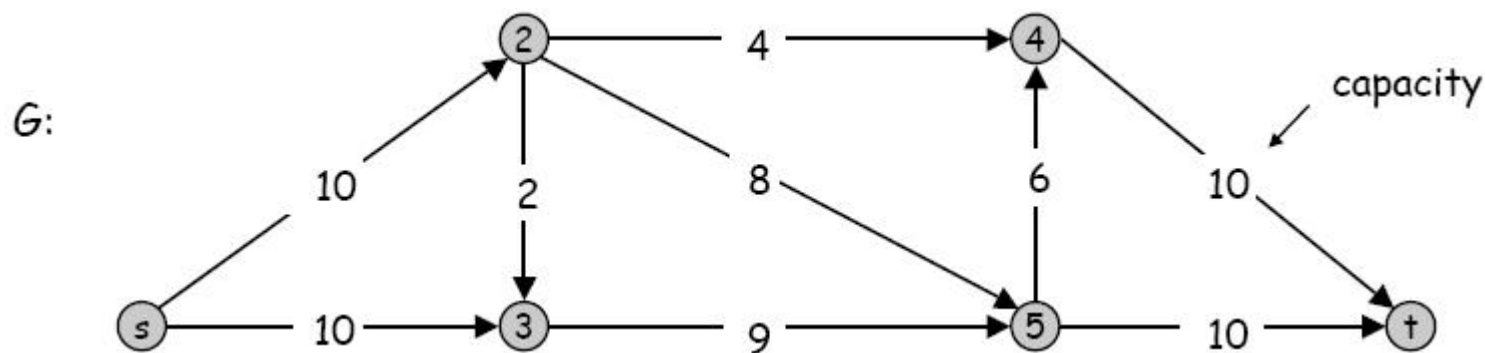
最小**s-t**割问题:

- 寻找一个最小容量的 **s-t** 割.



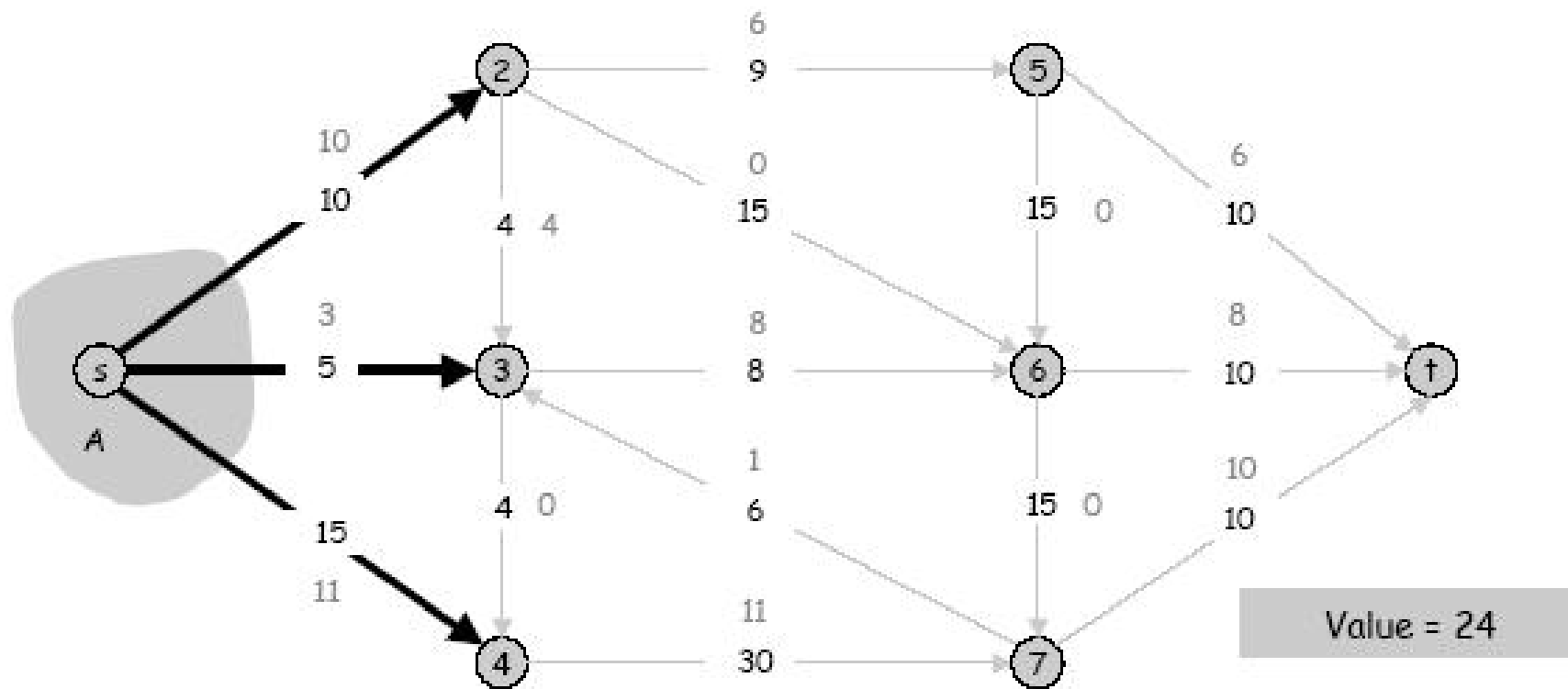
算法分析

- Demo演示(增广路径、瓶颈容量、剩余网络、 $f(x,y)$ 最大流)
- 初始图G

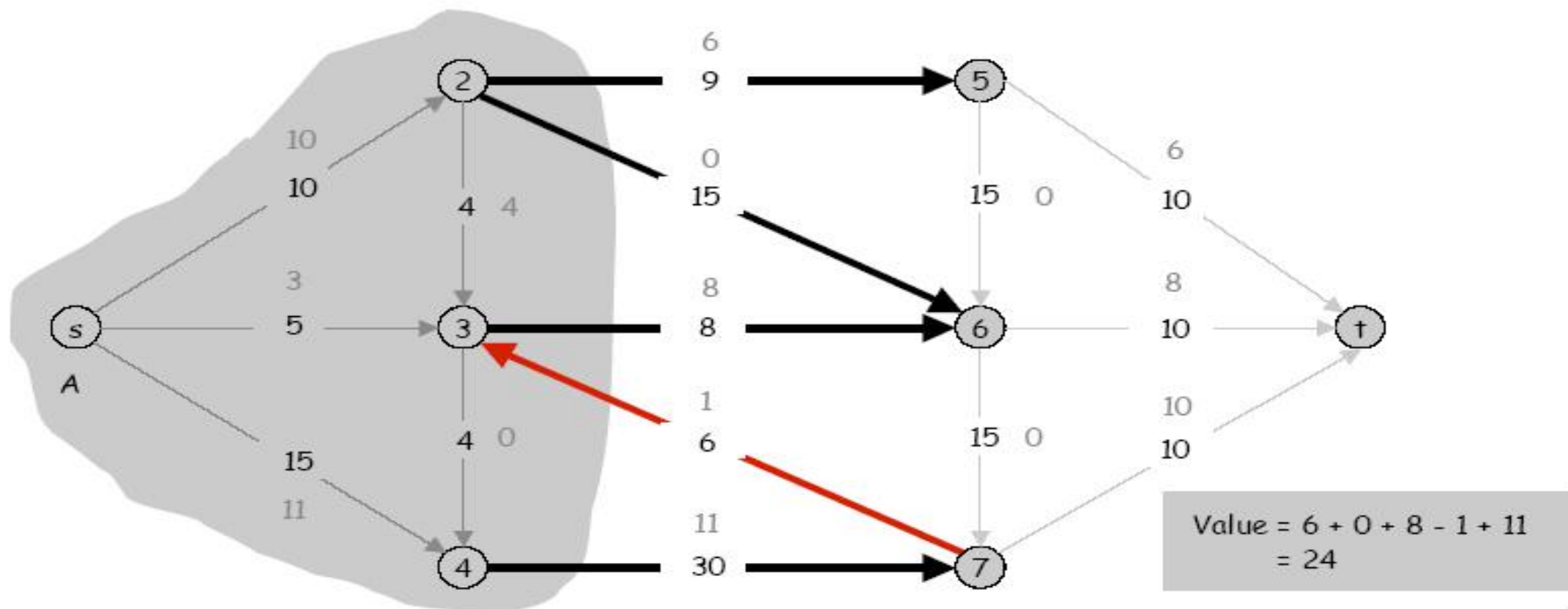


https://blog.csdn.net/qq_38662930/article/details/105242945

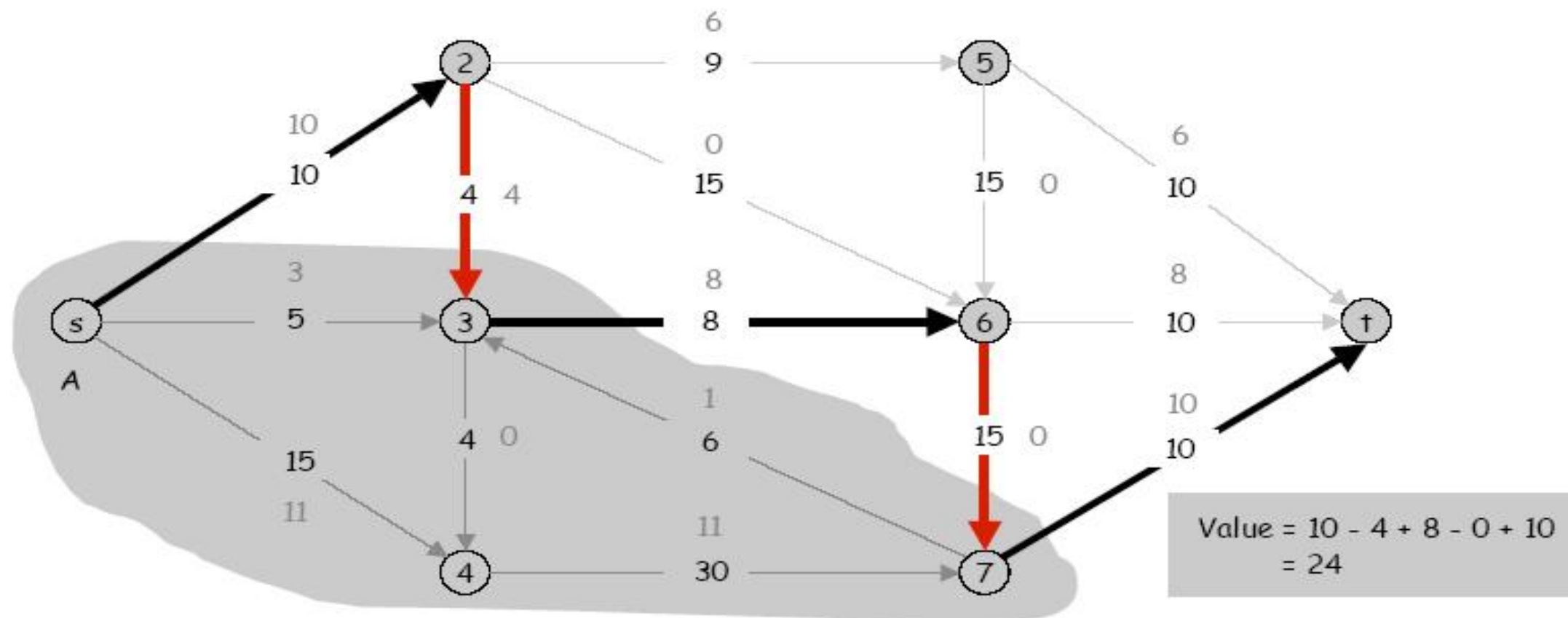
最大流与最小割



最大流与最小割



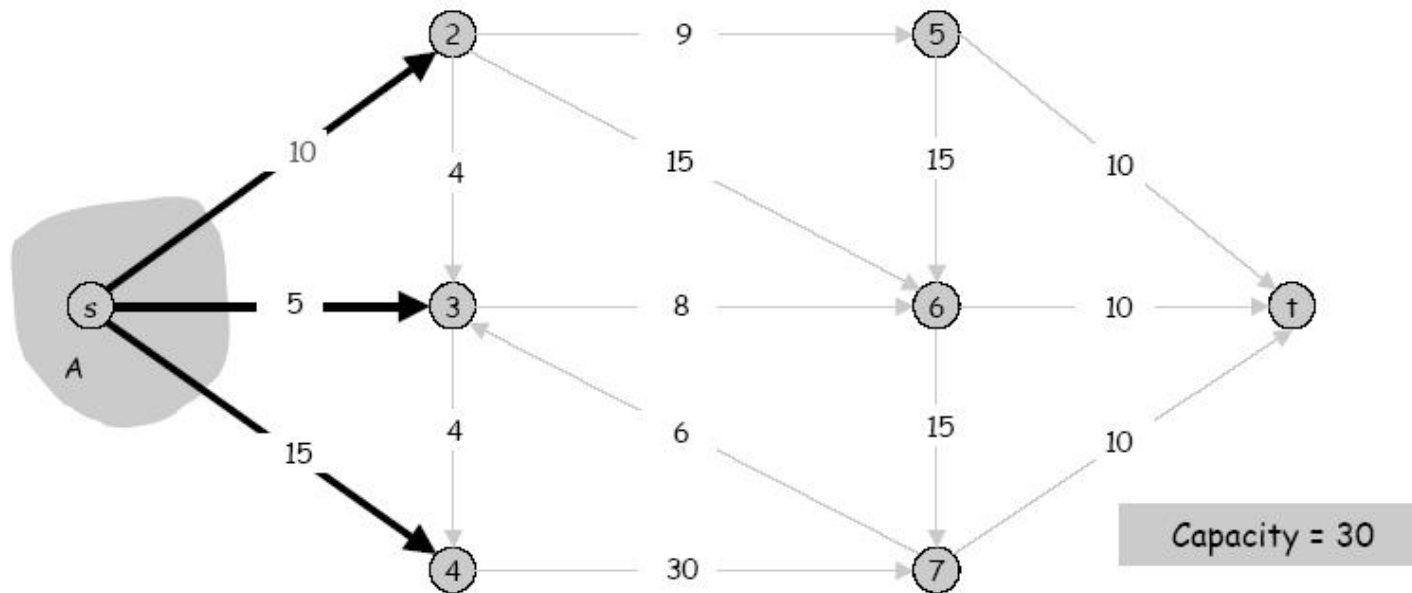
最大流与最小割



最大流与最小割

- 定理7.8 令 f 是任意 s - t 流, 且 (A,B) 是任意 s - t 割, 那么 $v(f) \leq c(A,B)$

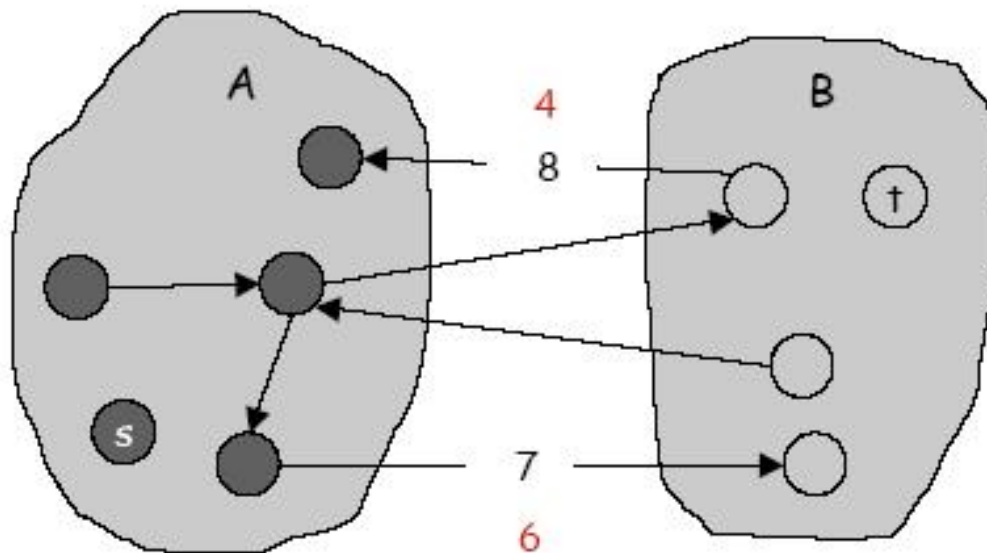
Cut capacity = 30 $\Rightarrow$ Flow value ≤ 30



最大流与最小割

■ 证明:

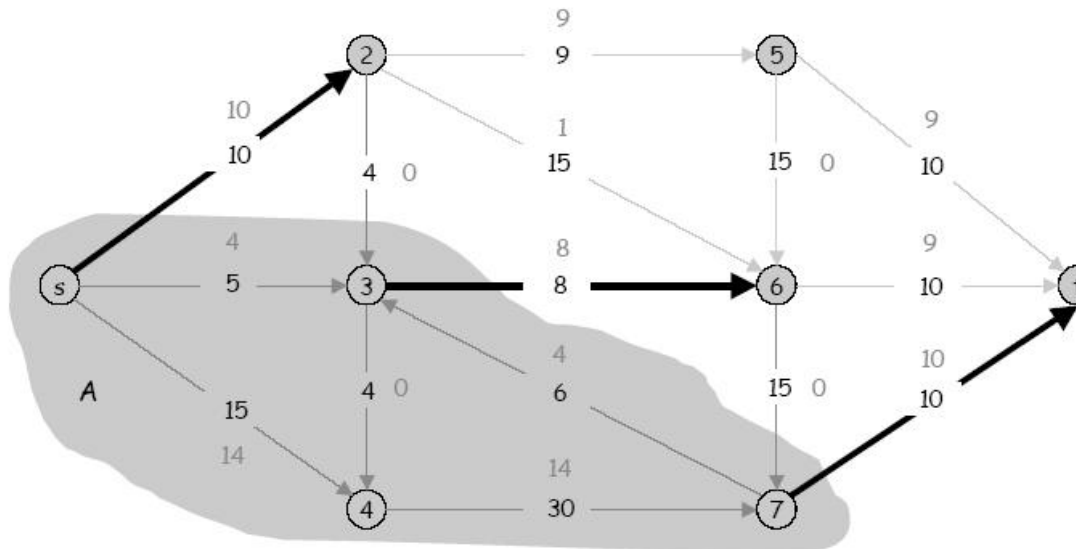
$$\begin{aligned} v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &\leq \sum_{e \text{ out of } A} f(e) \\ &\leq \sum_{e \text{ out of } A} c(e) \\ &= \text{cap}(A, B) \quad \blacksquare \end{aligned}$$

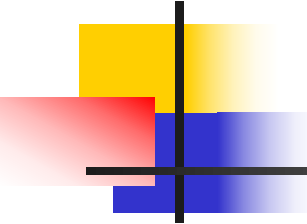


最大流与最小割

- 推论. 设 f 是任意的流, 并设 (A, B) 是任意的割. 如果 $v(f) = \text{cap}(A, B)$, 那么 f 是最大流, 并且 (A, B) 是最小割.

Value of flow = 28
Cut capacity = 28 $\Rightarrow$ Flow value $\leq$ 28





最大流与最小割

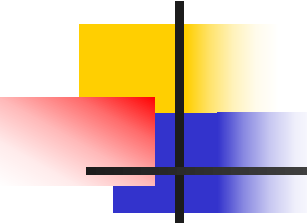
- 证明思路:

- (i) 存在 cut (A, B) 使得 $v(f) = \text{cap}(A, B)$.

- (ii) 流 f 是一个最大流.

- (iii) f 中没有增广路径.

(i) $\Rightarrow$ (ii) $\Rightarrow$ (iii) $\Rightarrow$ (i)



最大流与最小割

- (i) $\Rightarrow$ (ii) 显而易见
- (ii) $\Rightarrow$ (iii) 运用反证法. 设 f 是一个流, 如果还存在一条增广路经, 那么我们还可以继续改进 f , 矛盾。
- (iii) $\Rightarrow$ (i)

实际上这是算法停止运行的条件

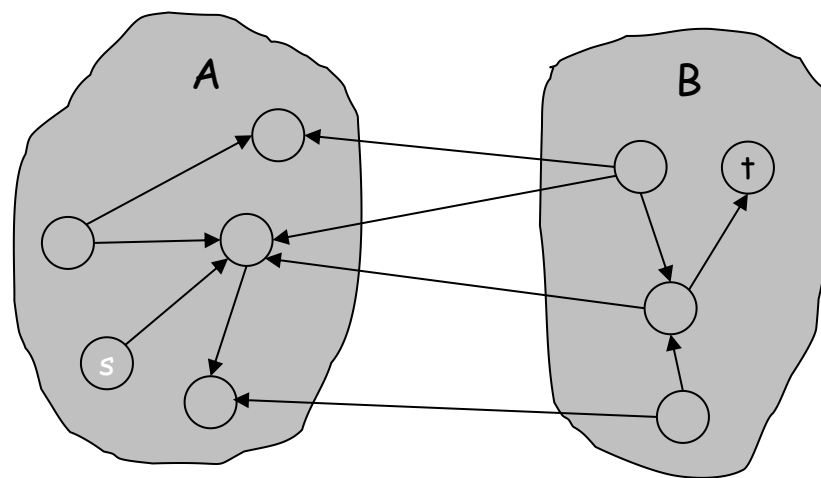
最大流与最小割

- 设流 f 没有增广路径.
- 定义集合 A 是剩余图 G_f 中从源点 s 可达顶点集合.
- 根据定义, 那么 $s \in A$; 终点 $t \notin A, t \in B$.

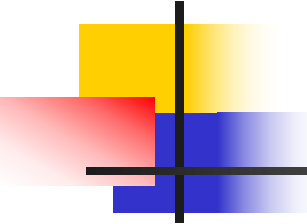
如果 $e=(u,v)$, $u \in A, v \in B$, 那么 $f(e)=c(e)$;

如果 $e'=(u',v')$, $u' \in B, v' \in A$, 那么 $f(e')=0$.

$$\begin{aligned} v(f) &= \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ in to } A} f(e) \\ &= \sum_{e \text{ out of } A} c(e) \\ &= \text{cap}(A, B) \end{aligned}$$

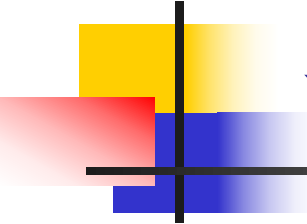


剩余图



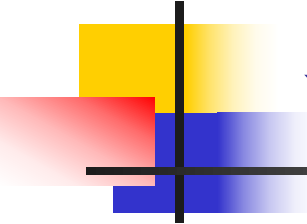
7.3 选择好的增广路径

- 一般的Ford-Fulkerson算法复杂度是不是输入规模的多项式时间？
(输入数据: $m, n, \log C$)
- 不是，定理7.5 告诉我们，Ford-Fulkerson算法的运行时间在 $O(mC)$ ，*伪多项式时间*
- 有时算法执行会非常没有效率



选择好的增广路径

- 之所以出现这样的问题，在于我们刚才选择了一条**瓶颈容量**很小的增广路径，导致**收敛**很慢
- 所以我们的思路是：
因为增广路径通过选择路径的瓶颈容量来增加最大流的值，我们选择**具有大的瓶颈容量的路径**，那么算法进展会更大些

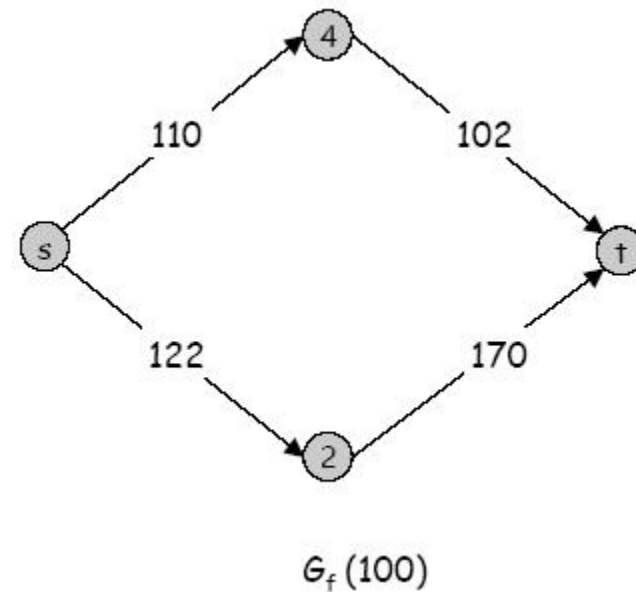
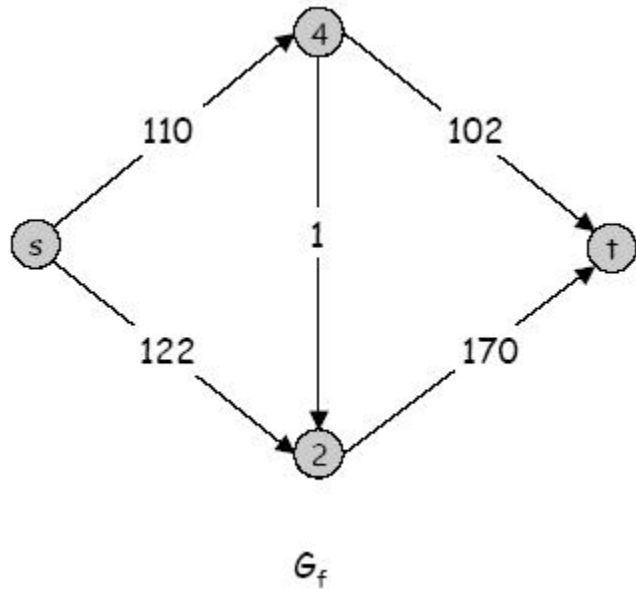


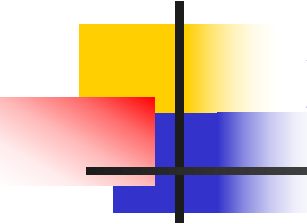
选择好的增广路径

- 选择好的增广路径:
Sufficiently large bottleneck capacity
- 这里为了便于控制, 我们维护一个称之为缩放参数的 Δ , 算法中将寻找瓶颈容量至少是 Δ 的路径。

选择好的增广路径

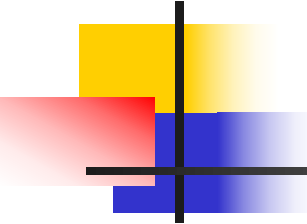
令 $G_f(\Delta)$ 是仅由剩余容量至少为 Δ 的边组成的剩余图的子集.





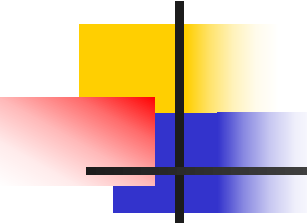
算法

```
Scaling-Max-Flow( $G, s, t, c$ ) {  
  foreach  $e \in E$   $f(e) \leftarrow 0$   
   $\Delta \leftarrow$  smallest power of 2 greater than or equal to  $C$   
   $G_f \leftarrow$  residual graph  
  
  while ( $\Delta \geq 1$ ) {  
     $G_f(\Delta) \leftarrow \Delta$ -residual graph  
    while (there exists augmenting path  $P$  in  $G_f(\Delta)$ ) {  
       $f \leftarrow$  augment( $f, c, P$ )  
      update  $G_f(\Delta)$   
    }  
     $\Delta \leftarrow \Delta / 2$   
  }  
  return  $f$   
}
```



算法分析

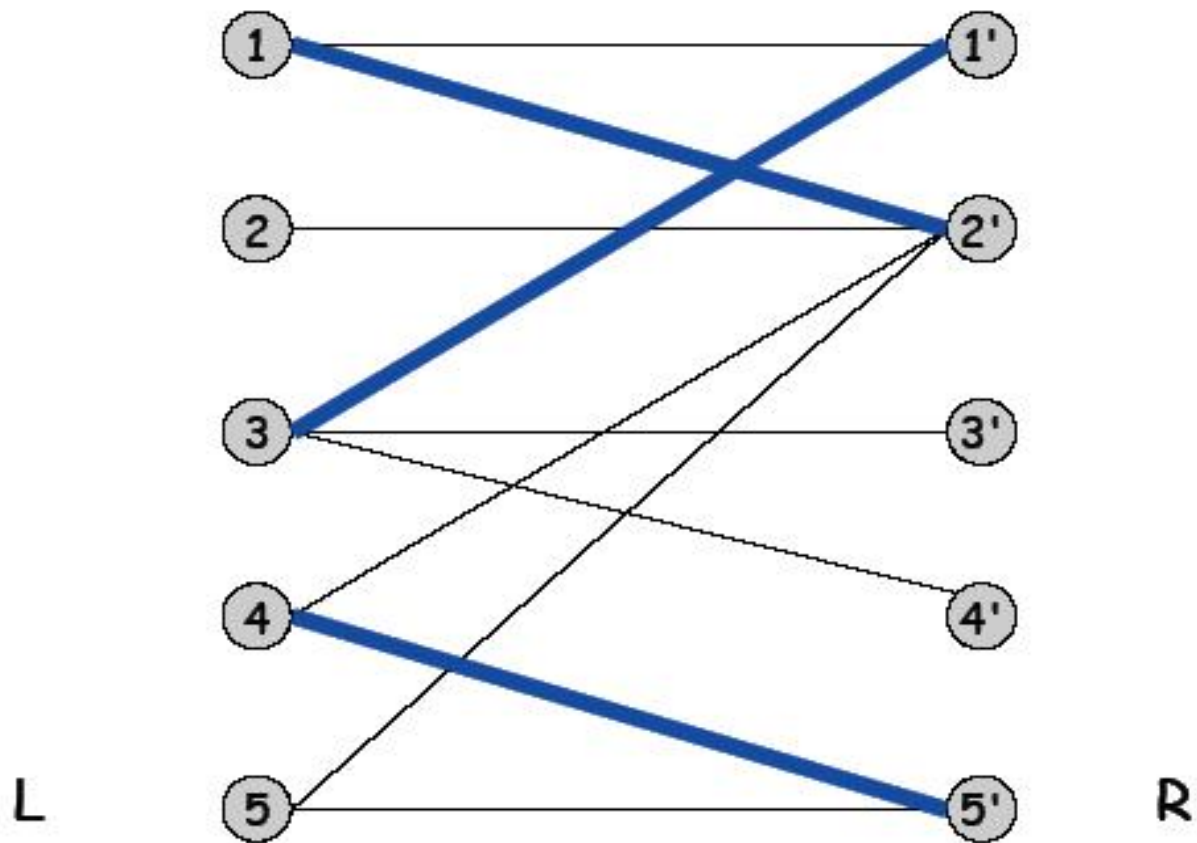
- 一次增广用 $O(m)$ 时间(包括建立图, 找到合适路径)
- 缩放次数: 至多 $1 + \lceil \log_2 C \rceil$
- 缩放阶段增广次数: 至多 $2m$
- 定理7.20 在具有 m 条边和整数容量的图中, 缩放最大流算法找最大流至多用 $2m(1 + \lceil \log_2 C \rceil)$ 次增广, 于是可在 $O(m^2 \log C)$ 时间内运行。



7.5 二分匹配问题

- 输入：无向图 $G = (V, E)$.
- $M \subseteq E$ 是一个匹配，如果每个结点至多出现在 M 中的一条边中。
- 最大匹配：寻找具有最大数目的匹配

二分匹配问题

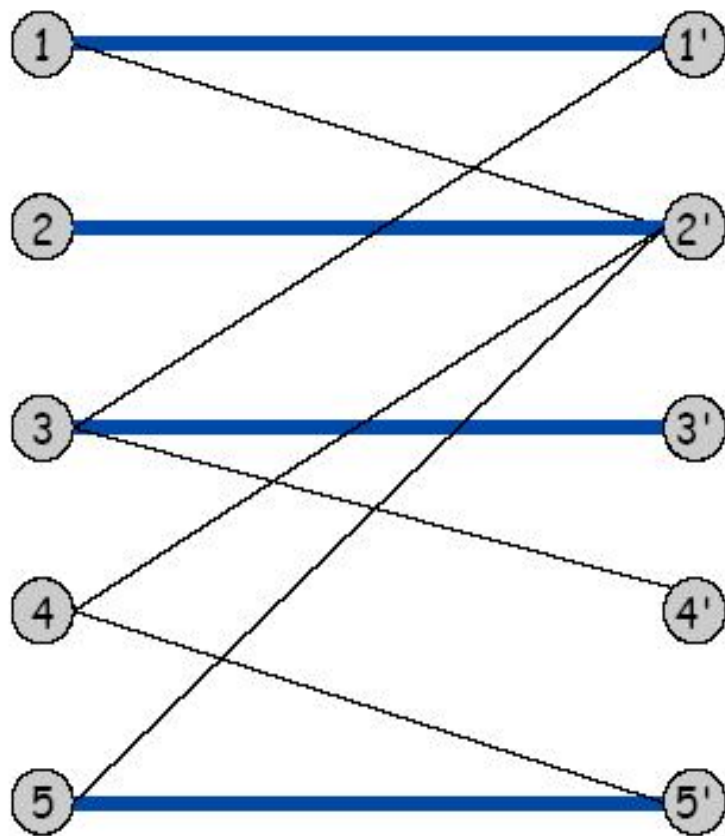


匹配

1-2', 3-1', 4-5'

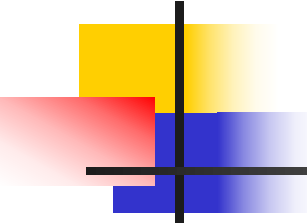
是最大匹配吗?

二分匹配问题



最大匹配

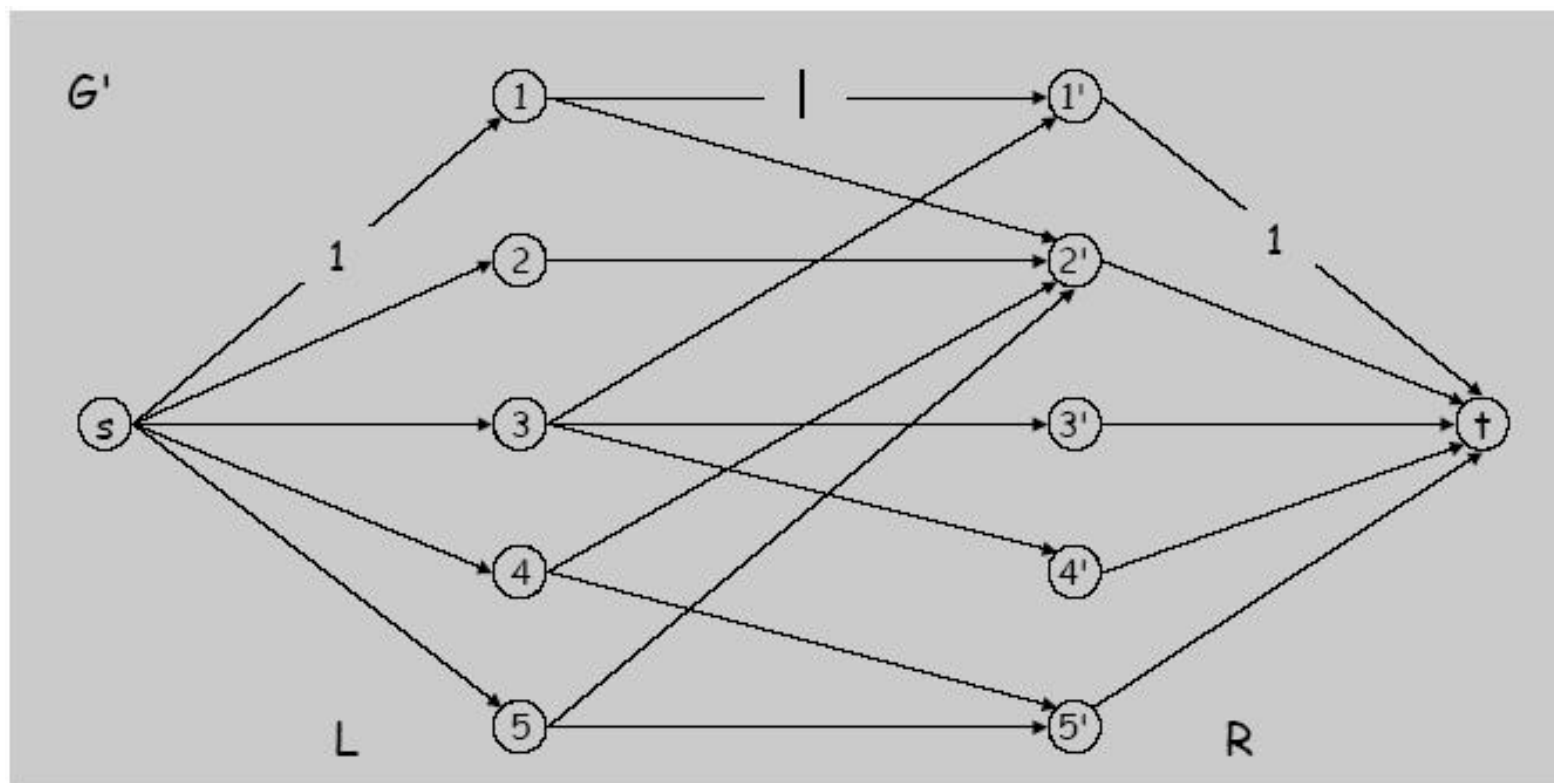
1-1', 2-2', 3-3' 4-4'



二分匹配问题

- 最大流的构造.
 - 构造图 $G' = (L \cup R \cup \{s, t\}, E')$.
 - 连接原图L到R的每条边, 每条边赋予单位容量.
 - 增加一个源点s, 从s到L中的每个结点连接一条边, 每条边赋予单位容量.
 - 增加一个终点 t, 从R中的每个结点到t连接一条边, 每条边赋予单位容量.

二分匹配问题

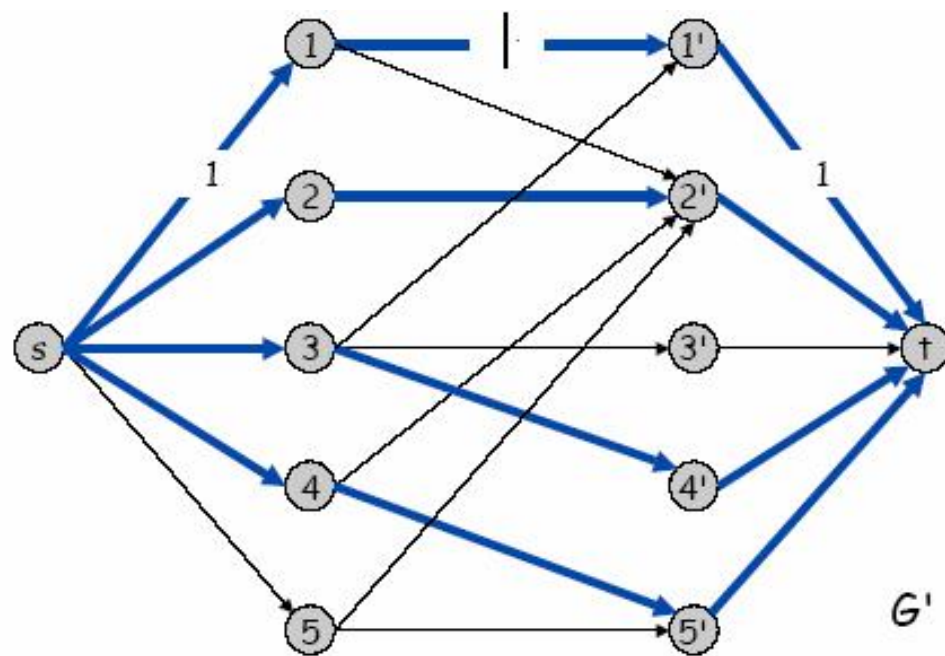
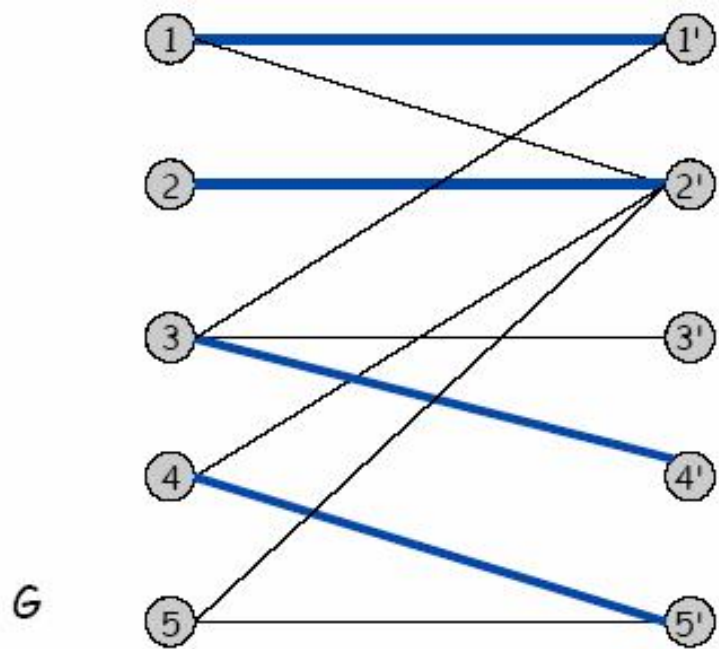


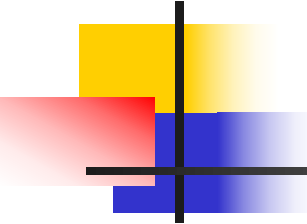


二分匹配问题

- 现在计算这个网络 G' 的最大 s - t 流，我们发现这个流的值等于 G 中最大匹配的大小。
- 定理. G 中最大匹配的数目与所定义的 G' 中最大流值相同.
- 证明:
 - 设 G 中最大匹配集合是 M ，其数目是 k .
 - 于是可以构造一个流 f ，每一条边从 s 出发，携带一个单位的容量.
 - f 是一个流，而且流值为 k .
 - 所以 G' 中最大流值 $\geq$ 最大匹配数目;

二分匹配问题





二分匹配问题

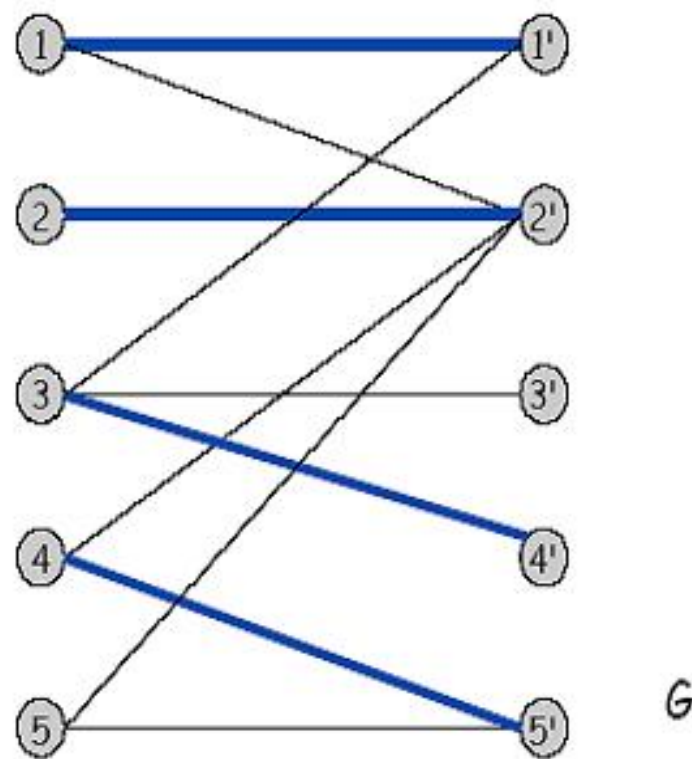
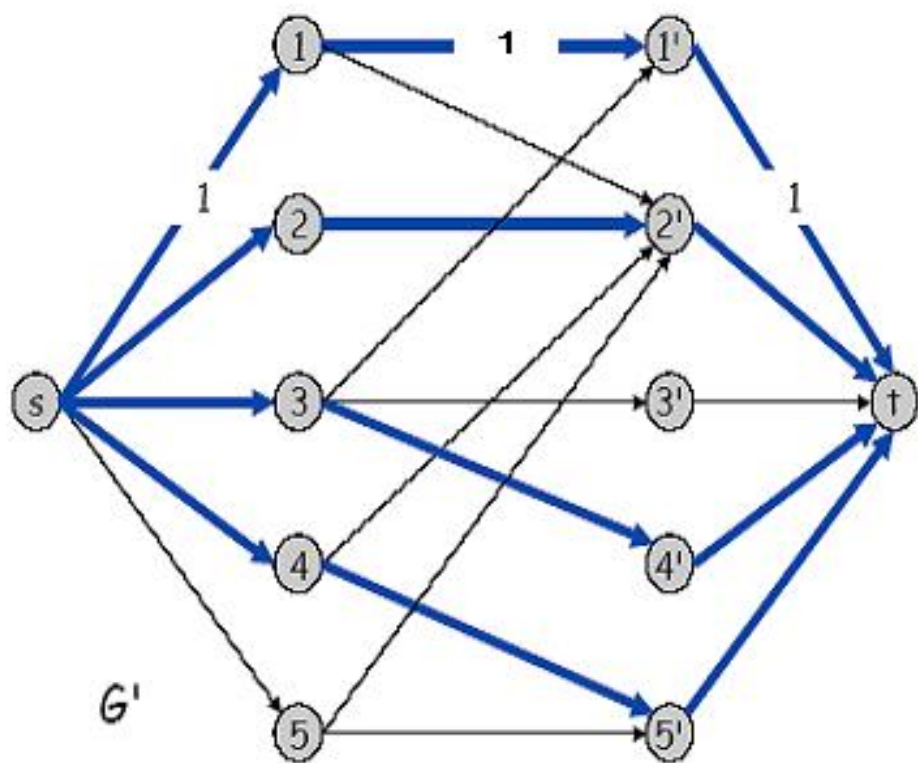
设 f 是 G' 最大流, 其值为 k .

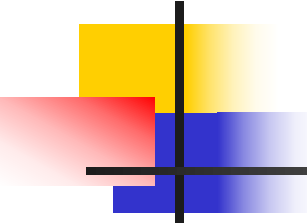
- 考虑集合 M : 从 L 到 R 权值为 1 的边的集合, i.e., $f(e) = 1$.
 - 可以发现每个节点至多在 M 的一条边中
 - $|M| = k$: 割 $(L \cup s, R \cup t)$ 就是一个匹配

所以最大匹配的数目 $\geq$ 最大流值

因此定理成立。

二分匹配问题





二分匹配问题： 界定运行时间

令 $n = |L| = |R|$, m 是 G 的边数

- 时间复杂度?
- 注意到 $C = |L| = n$, 根据以前 $O(mC)$ 的界
- 定理 7.38 可以用 **Ford-Fulkerson** 算法在 **$O(mn)$** 时间内找到二部图中的一个最大匹配。